



零資料接觸架構

讓 LLM 讀懂企業資料但永遠摸不到它

AI Enterprise Summit 2026 · 2026.07.02

Chester · Plusblocks Technology

Agenda



為什麼企業資料 × LLM 這麼難？

5 min



零資料接觸架構 (ZDTA) 設計

10 min



實作細節與 Demo

8 min



取捨、踩坑與決策框架

5 min



Q&A

2 min

01

為什麼企業資料 × LLM
這麼難？

企業的核心矛盾



想要

- LLM 理解我的資料
- 自然語言查詢報表
- 智慧化的業務洞察



害怕

- 資料送到外部模型
- Prompt injection 洩密
- 合規與稽核的噩夢

現有做法與瓶頸

RAG

Chunk 仍含原始資料



Fine-tuning

模型權重內化機密



On-prem LLM

成本高、能力受限



Data Masking

遮罩後語意喪失



共同問題：LLM 最終都會「接觸」到原始資料

02

零資料接觸架構

Zero Data Touch Architecture

核心理念

LLM 只看結構，不看內容



Schema-Only Context

只給 LLM 資料表結構、欄位名稱
與型別，不給任何一筆資料



Query Proxy

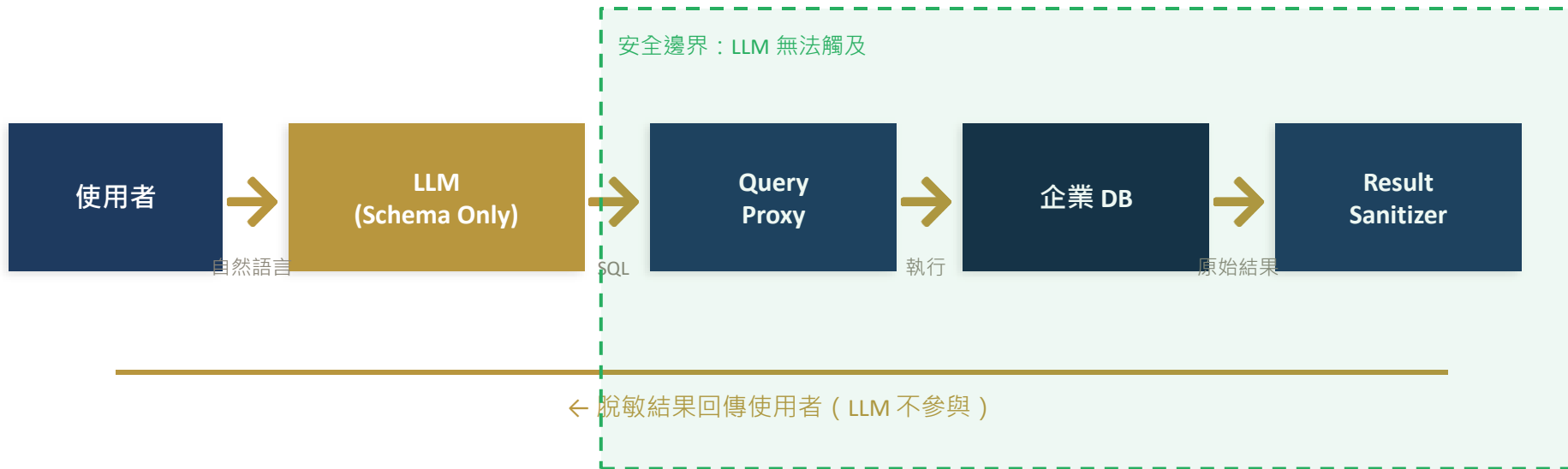
LLM 產生查詢語句，由安全中間
層代為執行，結果不回傳 LLM



Result Sanitizer

查詢結果經脫敏、聚合後才呈現
給使用者，LLM 全程不經手

架構總覽



Layer 1 : Schema-Only Context

LLM 收到的 Context

```
CREATE TABLE orders (  
  id          INT PK,  
  customer    VARCHAR(100),  
  amount      DECIMAL(10,2),  
  status      ENUM('pending',  
                   'shipped','done'),  
  created     TIMESTAMP  
);
```

LLM 看不到的

id	customer	amount
1	台積電	50M
2	鴻海	32M
3	聯發科	28M

✗ 任何一筆真實資料

Layer 2 : Query Proxy

LLM 產生 SQL → Proxy 驗證 → 安全執行

SQL 白名單驗證

僅允許 SELECT，禁止 INSERT / UPDATE / DELETE / DROP

欄位級權限控制

依使用者角色限制可查詢的欄位（如：業務看不到成本欄位）

查詢複雜度限制

限制 JOIN 深度、子查詢層數、結果筆數上限

Injection 防護

AST 解析而非字串匹配，阻擋 LLM 被誘導產生的惡意查詢

Layer 3 : Result Sanitizer

脫敏管線

1 PII 偵測與遮罩 (姓名、身分證、電話)

2 數值聚合 (個別金額 → 統計摘要)

3 K-Anonymity 驗證 (≥ 5 筆才回傳)

4 Audit Log 寫入 (誰查了什麼)



關鍵設計

查詢結果直接從 Sanitizer 送到前端，完全繞過 LLM。LLM 負責「理解意圖」和「產生查詢」，但永遠不經手任何一筆查詢結果。

LLM 的視野範圍

資訊類別	LLM 可見？	說明
Table Schema	✓ 可見	欄位名、型別、關聯
欄位說明 / 註解	✓ 可見	幫助 LLM 理解業務語意
Enum / 狀態值	✓ 可見	讓 WHERE 子句正確
實際資料列	✗ 不可見	全程隔離
查詢結果	✗ 不可見	直送前端，不經 LLM
使用者身分	⚠ 部分	僅角色，不含 PII

03

實作細節與 Demo

技術堆疊

Schema Extractor

pg_dump --schema-only + 自訂 annotation parser

LLM Layer

Claude API (Function Calling + System Prompt 注入 Schema)

Query Proxy

Bun + TypeScript + node-sql-parser (AST 驗證)

Sanitizer

Presidio (PII) + 自訂 K-Anonymity 模組

Audit & Logging

結構化 JSON log → Loki / Grafana

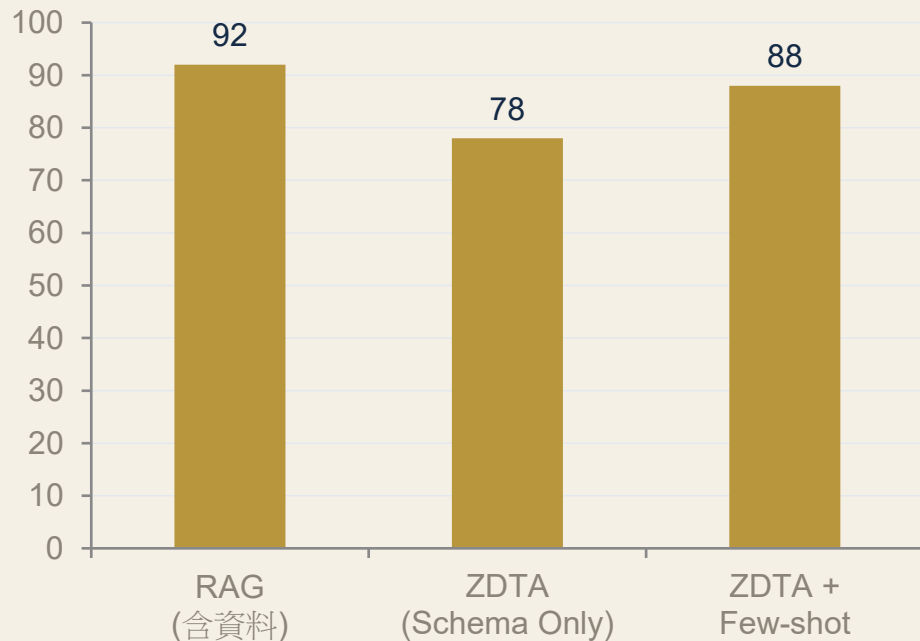
Frontend

React + Vite , 結果直接從 Sanitizer WebSocket 推送

Demo：金融場景

- 1 使用者問： 「上季前十大客戶的交易總額趨勢？」
- 2 LLM 產生： `SELECT customer_id, SUM(amount)... GROUP BY... ORDER BY... LIMIT 10`
- 3 Proxy 驗證： ☒ SELECT-only ☒ 無禁止欄位 ☒ 結果數 ≤ 上限
- 4 DB 執行： 回傳原始結果 → Sanitizer
- 5 Sanitizer： 客戶名稱以代碼呈現，金額聚合為區間，≥ 5 筆
- 6 前端呈現： 使用者看到脫敏圖表，LLM 全程未見任何資料

準確度 vs 安全性的取捨



Few-shot 秘訣

- 提供「問題 → SQL」配對範例
- 範例中不含真實資料
- 涵蓋常見查詢模式
- 定期依使用者回饋更新

04

取捨、踩坑 與決策框架

踩過的坑



Schema 太大，Context 爆了

→ 動態 Schema 選取：依使用者問題只注入相關表格



LLM 幻覺產生不存在的欄位

→ Proxy 的 AST 驗證會攔截，但需要好的錯誤訊息引導 retry



聚合過度，結果失去意義

→ 建立「最小脫敏規則」，依資料敏感等級分層處理



延遲增加 ~800ms

→ Schema cache + Query plan cache，降至 ~200ms 額外延遲

何時該用 ZDTA ?

適合場景

- 結構化資料 (SQL 資料庫)
- 高度監管行業 (金融、醫療)
- 查詢型需求為主
- 已有完善的 Schema 文件
- 需要完整 Audit trail

不太適合

- 非結構化資料 (文件、圖片)
- 需要 LLM 做深度推理
- 資料量小且不敏感
- 沒有明確 Schema
- 需要即時對話式分析

Key Takeaways

- ✓ LLM 不需要看到資料，也能理解資料 — Schema 就是語言
- ✓ 安全邊界要在架構層實現，而非靠 Prompt 約束
- ✓ 準確度的損失可以用 Few-shot 和動態 Schema 補回來
- ✓ 每一筆查詢都要有 Audit trail — 合規的基石

?

Q & A

Chester · Plusblocks Technology

感謝聆聽